

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет <факультет>

Кафедра <кафедра>

Дисциплина: Информационные технологии и веб-программирование

ОТЧЁТ
по лабораторной работе 3
на тему

РЕАЛИЗАЦИЯ СИСТЕМЫ АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ

Студент: <И.О. Фамилия студента>

Руководитель: <И.О. Фамилия руководителя>

Минск 2026

ЦЕЛЬ РАБОТЫ

Освоить механизмы аутентификации и авторизации в веб-приложениях: реализовать регистрацию и вход пользователей на основе JSON Web Token, хеширование паролей с помощью bcrypt и защиту маршрутов с помощью middleware в серверной части проекта «Платформа для управления мероприятиями».

ПОСТАНОВКА ЗАДАЧИ

Работа продолжает лабораторные работы 1 и 2: используется приложение на Express.js и ORM Sequelize с базой данных PostgreSQL, в которой уже хранятся мероприятия. Задание предполагает выполнение следующих подзадач:

- создать модель пользователя с адресом электронной почты и хешем пароля;
- реализовать маршрут регистрации с проверкой уникальности адреса и хешированием пароля;
- реализовать маршрут входа с проверкой пароля и выдачей токена;
- создать middleware, проверяющий токен в заголовке Authorization;
- защитить маршруты, доступные только авторизованным пользователям, и добавить маршрут получения данных текущего пользователя;
- реализовать дополнительный механизм безопасности.

В качестве дополнительного механизма выбрана ролевая модель доступа. Для платформы управления мероприятиями она естественна: просматривать мероприятия может любой посетитель, а создавать, изменять и удалять их должен только администратор.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Аутентификация и авторизация

Аутентификация отвечает на вопрос, кто обращается к системе, и подтверждается, например, паролем. Авторизация определяет, что именно разрешено уже опознанному пользователю. Сначала выполняется первая процедура, затем вторая: сервер не может решить, доступен ли ресурс, пока не знает, кто к нему обращается.

1.2 Хеширование паролей

Пароли нельзя хранить в открытом виде: при утечке базы данных злоумышленник получил бы доступ к учётным записям. Вместо пароля хранится его хеш, по которому исходное значение восстановить нельзя. Алгоритм bcrypt добавляет к паролю случайную соль и многократно повторяет вычисление, число повторов задаётся параметром стоимости. Соль делает одинаковые пароли разными хешами и защищает от атак по радужным таблицам, а стоимость замедляет перебор. В работе используется стоимость 10.

1.3 Структура JWT

JSON Web Token состоит из трёх частей, разделённых точками: заголовок с алгоритмом подписи, полезной нагрузки с утверждениями (идентификатор пользователя, адрес почты, роль, время выдачи и срок действия) и подписи. Подпись вычисляется по первым двум частям с помощью секретного ключа, известного только серверу. Чтобы проверить подлинность токена, сервер заново вычисляет подпись и сравнивает её с полученной, а также проверяет, что срок действия не истёк. Нагрузка кодируется, но не шифруется, поэтому секретные данные в токен помещать нельзя.

1.4 Передача токена и защита маршрутов

Клиент передаёт токен в заголовке Authorization в формате Bearer. Middleware извлекает токен, проверяет его и прикрепляет расшифрованную нагрузку к объекту запроса, после чего передаёт управление следующему обработчику. Если токена нет или он недействителен, возвращается код 401. Если пользователь опознан, но прав у него недостаточно, возвращается код 403.

1.5 Ролевая модель доступа

В ролевой модели права привязаны не к отдельному пользователю, а к роли. У пользователя есть поле с ролью, а отдельный middleware сравнивает роль из токена с требуемой и пропускает запрос только при совпадении.

2 ХОД РАБОТЫ

2.1 Установка зависимостей и настройка окружения

В проект были добавлены пакеты `jsonwebtoken` и `bcrypt`. Секретный ключ подписи токенов и строка подключения к базе данных вынесены в переменные окружения, пример значений приведён в файле `.env.example`, а сам файл `.env` исключён из репозитория.

2.2 Модель пользователя

Модель `User` содержит первичный ключ, адрес электронной почты с признаком уникальности, хеш пароля и роль. По умолчанию роль равна `user`, допустимые значения ограничены списком `user` и `admin`. Листинг модели приведён ниже.

Листинг 2.1 – Модель `User`

```
'use strict';
const {
  Model
} = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class User extends Model {
    static associate(models) {
      // define association here
    }
  }
  User.init({
    email: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: true
    },
    passwordHash: {
      type: DataTypes.STRING,
      allowNull: false
    },
    role: {
      type: DataTypes.STRING,
      allowNull: false,
      defaultValue: 'user',
      validate: { isIn: [['user', 'admin']] }
    }
  }, {
    sequelize,
    modelName: 'User',
  });
  return User;
};
```

Структура таблицы создаётся миграцией, листинг которой приведён ниже.

Листинг 2.2 – Миграция создания таблицы Users

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('Users', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      email: {
        type: Sequelize.STRING,
        allowNull: false,
        unique: true
      },
      passwordHash: {
        type: Sequelize.STRING,
        allowNull: false
      },
      role: {
        type: Sequelize.STRING,
        allowNull: false,
        defaultValue: 'user'
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface) {
    await queryInterface.dropTable('Users');
  }
};
```

2.3 Регистрация и вход

Контроллер регистрации проверяет наличие обязательных полей, ищет пользователя с таким же адресом и при совпадении возвращает код 409. Затем пароль хешируется функцией `bcrypt.hash` со стоимостью 10, пользователь сохраняется, и клиент получает код 201 с идентификатором, адресом и

ролью. Хеш в ответ не включается. Маршрут входа находит пользователя, сравнивает пароль с хешем и выдаёт токен со сроком действия один час, в который записаны идентификатор, адрес и роль. При неверном адресе или пароле возвращается одинаковое сообщение с кодом 401, чтобы нельзя было узнать, существует ли такой адрес. Листинг контроллера приведён ниже.

Листинг 2.3 – Контроллер аутентификации

```
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const { User } = require('../models');

const register = async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: "Укажите email и
    ↪ password" });
  }

  try {
    const existing = await User.findOne({ where: { email }
    ↪ });
    if (existing) {
      return res.status(409).json({ message: "Пользователь
      ↪ с таким email уже существует" });
    }

    const passwordHash = await bcrypt.hash(password, 10);
    const user = await User.create({ email, passwordHash });

    res.status(201).json({ id: user.id, email: user.email,
    ↪ role: user.role });
  } catch (error) {
    res.status(500).json({ message: "Ошибка при регистрации",
    ↪ error: error.message });
  }
};

const login = async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: "Укажите email и
    ↪ password" });
  }

  try {
    const user = await User.findOne({ where: { email } });
    const valid = user && await bcrypt.compare(password,
    ↪ user.passwordHash);
    if (!valid) {
```

```

        return res.status(401).json({ message: "Неверный
        ↪ email или пароль" });
    }

    const token = jwt.sign(
        { id: user.id, email: user.email, role: user.role },
        process.env.JWT_SECRET,
        { expiresIn: '1h' }
    );

    res.status(200).json({ token });
} catch (error) {
    res.status(500).json({ message: "Ошибка при входе",
    ↪ error: error.message });
}
};

const getProfile = async (req, res) => {
    try {
        const user = await User.findByPk(req.user.id, {
            attributes: ['id', 'email', 'role', 'createdAt']
        });
        if (!user) {
            return res.status(404).json({ message: "Пользователь
            ↪ не найден" });
        }
        res.status(200).json(user);
    } catch (error) {
        res.status(500).json({ message: "Ошибка сервера", error:
        ↪ error.message });
    }
};

const getUsers = async (req, res) => {
    try {
        const users = await User.findAll({ attributes: ['id',
        ↪ 'email', 'role', 'createdAt' ] });
        res.status(200).json(users);
    } catch (error) {
        res.status(500).json({ message: "Ошибка сервера", error:
        ↪ error.message });
    }
};

module.exports = { register, login, getProfile, getUsers };

```

Результат успешной регистрации показан на рисунке 2.1, а попытка повторной регистрации того же адреса на рисунке 2.2.

```
> node E:/lab3-demo/demo.js register
POST /auth/register
{"email":"student@example.com","password":"Secret123"}
HTTP 201 Created
{
  "id": 1,
  "email": "student@example.com",
  "role": "user"
}
```

Рисунок 2.1 – Успешная регистрация пользователя

```
> node E:/lab3-demo/demo.js duplicate
POST /auth/register
{"email":"student@example.com","password":"Secret123"}
HTTP 201 Created
{
  "id": 1,
  "email": "student@example.com",
  "role": "user"
}

POST /auth/register
{"email":"student@example.com","password":"Secret123"}
HTTP 409 Conflict
{
  "message": "Пользователь с таким email уже существует"
}
```

Рисунок 2.2 – Повторная регистрация и отказ с кодом 409

Вход с верными данными возвращает токен, вход с неверным паролем завершается отказом, как показано на рисунке 2.3.

```
> node E:/lab3-demo/demo.js login
POST /auth/login
{"email":"student@example.com","password":"Secret123"}
HTTP 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}

POST /auth/login
{"email":"student@example.com","password":"wrong"}
HTTP 401 Unauthorized
{
  "message": "Неверный email или пароль"
}
```

Рисунок 2.3 – Вход пользователя и получение токена

2.4 Middleware проверки токена

Функция `authenticate` выделяет токен из заголовка `Authorization`, проверяет схему `Bearer`, верифицирует подпись и срок действия и сохраняет расшифрованные данные в `req.user`. Функция `isAdmin` пропускает запрос только при роли `admin`. Листинг приведён ниже.

Листинг 2.4 – Middleware аутентификации и проверки роли

```
const jwt = require('jsonwebtoken');

const authenticate = (req, res, next) => {
  const header = req.headers.authorization || '';
  const [scheme, token] = header.split(' ');

  if (scheme !== 'Bearer' || !token) {
    return res.status(401).json({ message: "Требуется  
→ авторизация: заголовок Authorization: Bearer <token>"  
→ });
  }

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (error) {
    res.status(401).json({ message: "Недействительный или  
→ просроченный токен" });
  }
};

const isAdmin = (req, res, next) => {
  if (!req.user || req.user.role !== 'admin') {
    return res.status(403).json({ message: "Доступ только для  
→ администраторов" });
  }
  next();
};

module.exports = { authenticate, isAdmin };
```

2.5 Защищённые маршруты

Маршрут GET /profile подключён через authenticate и возвращает данные текущего пользователя, взятые из базы по идентификатору из токена, без хеша пароля. Запрос с действительным токеном показан на рисунке 2.4, а запросы без токена и с недействительным токеном, завершающиеся кодом 401, на рисунке 2.5.

Листинг 2.5 – Маршруты профиля и списка пользователей

```
const express = require('express');
const router = express.Router();

const { authenticate, isAdmin } = require('../middleware/auth');
const { getProfile, getUsers } =
  → require('../controllers/authController');
```

```
router.get('/profile', authenticate, getProfile);
router.get('/users', authenticate, isAdmin, getUsers);

module.exports = router;
```

```
> node E:/lab3-demo/demo.js profile
POST /auth/login
{"email":"student@example.com","password":"Secret123"}
HTTP 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}

GET /profile (Authorization: Bearer <token>)
HTTP 200 OK
{
  "id": 1,
  "email": "student@example.com",
  "role": "user",
  "createdAt": "2026-10-07T09:53:59.811Z"
}
```

Рисунок 2.4 – Запрос профиля с действительным токеном

```
> node E:/lab3-demo/demo.js noauth
GET /profile
HTTP 401 Unauthorized
{
  "message": "Требуется авторизация: заголовок Authorization: Bearer <token>"
}

GET /profile (Authorization: Bearer <token>)
HTTP 401 Unauthorized
{
  "message": "Недействительный или просроченный токен"
}
```

Рисунок 2.5 – Запросы к защищённому маршруту без токена и с недействительным токеном

2.6 Дополнительный механизм: ролевая модель

Поле `role` добавлено в модель и миграцию, роль записывается в токен при входе. Изменяющие операции над мероприятиями (создание, обновление, удаление) закрыты цепочкой `authenticate` и `isAdmin`, чтение списка и отдельного мероприятия осталось открытым. Дополнительно администратору доступен маршрут `GET /users` со списком пользователей. Листинг маршрутов мероприятий приведён ниже.

Листинг 2.6 – Маршруты мероприятий с проверкой роли

```

const express = require('express');
const router = express.Router();

const {
  getEvents,
  getEventById,
  createEvent,
  updateEvent,
  deleteEvent
} = require('../controllers/eventController');
const { authenticate, isAdmin } = require('../middleware/auth');

router.get('/', getEvents);
router.get('/:id', getEventById);
router.post('/', authenticate, isAdmin, createEvent);
router.put('/:id', authenticate, isAdmin, updateEvent);
router.delete('/:id', authenticate, isAdmin, deleteEvent);

module.exports = router;

```

Новые пользователи получают роль `user`, а роль `admin` назначается напрямую в базе данных, чтобы нельзя было стать администратором через публичный маршрут регистрации. На рисунке 2.6 показано, что обычный пользователь при попытке создать мероприятие получает код 403, а администратор код 201.

```

> node E:/lab3-demo/demo.js rbac
POST /events (Authorization: Bearer <token>)
{"title":"Вебинар по JWT","type":"webinar","date":"2026-11-20"}
HTTP 403 Forbidden
{
  "message": "Доступ только для администраторов"
}

POST /events (Authorization: Bearer <token>)
{"title":"Вебинар по JWT","type":"webinar","date":"2026-11-20"}
HTTP 201 Created
{
  "id": 1,
  "title": "Вебинар по JWT",
  "type": "webinar",
  "date": "2026-11-20T00:00:00.000Z",
  "updatedAt": "2026-10-07T09:54:03.318Z",
  "createdAt": "2026-10-07T09:54:03.318Z"
}

```

Рисунок 2.6 – Создание мероприятия пользователем и администратором

2.7 Автоматические тесты

Для проверки работы написаны автоматические тесты на `jest` и `supertest`. Обращения к базе данных в них заменены заглушками, поэтому тесты не зависят от внешнего сервера. Проверяются регистрация, вход, срок

действия и подпись токена, ответ профиля, разграничение доступа по ролям для всех изменяющих маршрутов мероприятий, обработка ошибок и определения моделей. Всего выполняется 51 тест, покрытие строк кода составляет 98,6 %. Результат запуска показан на рисунке 2.7.

```
> cd /d E:/event-management && npx jest --coverage
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	98.6	97.43	94.44	98.6	
event-management	90	50	66.66	90	
server.js	90	50	66.66	90	26-27
event-management/controllers	100	100	100	100	
authController.js	100	100	100	100	
eventController.js	100	100	100	100	
event-management/middleware	100	100	100	100	
auth.js	100	100	100	100	
event-management/models	100	100	100	100	
event.js	100	100	100	100	
user.js	100	100	100	100	
event-management/routes	100	100	100	100	
auth.js	100	100	100	100	
eventRoutes.js	100	100	100	100	
profile.js	100	100	100	100	

```

Test Suites: 3 passed, 3 total
Tests:       51 passed, 51 total
Snapshots:   0 total
Time:        3.409 s
Ran all test suites.

```

Рисунок 2.7 – Результат выполнения автоматических тестов

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

- Аутентификация устанавливает личность пользователя, авторизация определяет его права на действия и ресурсы. Первая предшествует второй;
- JWT состоит из заголовка, полезной нагрузки и подписи. Сервер проверяет подлинность, вычисляя подпись по заголовку и нагрузке секретным ключом и сравнивая её с полученной, а также проверяя срок действия;
- Хеш защищает пароли при утечке базы данных, потому что по нему нельзя восстановить пароль. Соль это случайная строка, добавляемая к паролю перед хешированием: она делает хеши одинаковых паролей различными и лишает смысла готовые таблицы хешей;
- Безопаснее httpOnly cookie с флагами Secure и SameSite: скрипт страницы не может прочитать такое значение. Токен в localStorage доступен любому скрипту на странице. Cookie, в свою очередь, требует защиты от CSRF;
- Маршрут защищают цепочкой middleware: сначала проверка токена, затем проверка роли, например isAdmin. При недостатке прав возвращается код 403;
- Токен хранится у клиента, поэтому выход сводится к удалению токена на клиенте. Для немедленного отзыва на сервере используют чёрный список идентификаторов токенов или счётчик версий токена, короткий срок жизни и refresh-токены;
- При хранении в localStorage уязвимость XSS позволяет внедрённому скрипту прочитать токен и отправить его злоумышленнику, который затем действует от имени пользователя до окончания срока токена;
- Выдаются два токена: короткоживущий access-токен и долгоживущий refresh-токен. Когда срок access-токена истекает, клиент отправляет refresh-токен на специальный маршрут и получает новую пару. Refresh-токен хранится на сервере и может быть отозван;
- Для каждой выданной пары создаётся запись сессии, привязанная к пользователю. При входе сервер считает активные сессии и при превышении лимита отказывает во входе или завершает самую старую. Токен содержит идентификатор сессии, который проверяется при каждом запросе;
- Ограничивают число попыток входа с одного адреса и для одной учётной записи, вводят задержки и временную блокировку, применяют лимитирование запросов, требуют сложные пароли и дополнительный фактор;
- Тот, кто знает ключ, может подписывать любые токены и выдавать себя за любого пользователя, включая администратора. Ключ необходимо немедленно заменить, при этом все ранее выданные токены перестанут проходить проверку, а пользователям придётся войти заново;
- В записи пользователя хранят число неудачных попыток и время окончания блокировки. После заданного числа ошибок подряд, например пяти, в поле блокировки записывается момент через пять минут, и пока он не наступил, вход отклоняется. При успешном входе счётчик обнуляется;

- Токен обычно передаётся в заголовке Authorization в формате «Bearer» и самого токена через пробел;
- Срок задаётся параметром expiresIn при подписании, например один час. Он записывается в утверждение exp, и при проверке библиотека сама отклоняет просроченный токен;
- До истечения срока токен без дополнительных мер отозвать нельзя, потому что сервер ничего не хранит. Отзыв возможен через чёрный список идентификаторов, хранение версии токена в записи пользователя, смену ключа подписи и короткий срок жизни в сочетании с refresh-токенами;
- Сессии с cookie хранят состояние на сервере: их легко отозвать, но они хуже масштабируются и требуют общего хранилища. JWT не требует хранения состояния и удобен для API и мобильных клиентов, но отзыв затруднён, а токен нужно беречь. Для классических сайтов с одним сервером подходят сессии, для распределённых систем и API чаще выбирают JWT.

ВЫВОДЫ

В ходе работы реализована система аутентификации и авторизации серверной части платформы управления мероприятиями: пользователи регистрируются, пароли хранятся в виде хешей bcrypt, при входе выдаётся токен JWT со сроком действия один час, а middleware проверяет токен и защищает маршруты. Получение данных текущего пользователя доступно только по действительному токenu, запросы без токена отклоняются с кодом 401.

Дополнительно внедрена ролевая модель доступа: изменение мероприятий разрешено только администратору, остальным возвращается код 403. Автоматические тесты подтверждают работу всех маршрутов и разграничения прав. Поставленная цель достигнута, реализованный механизм будет использован при интеграции клиентского приложения в следующей лабораторной работе.